

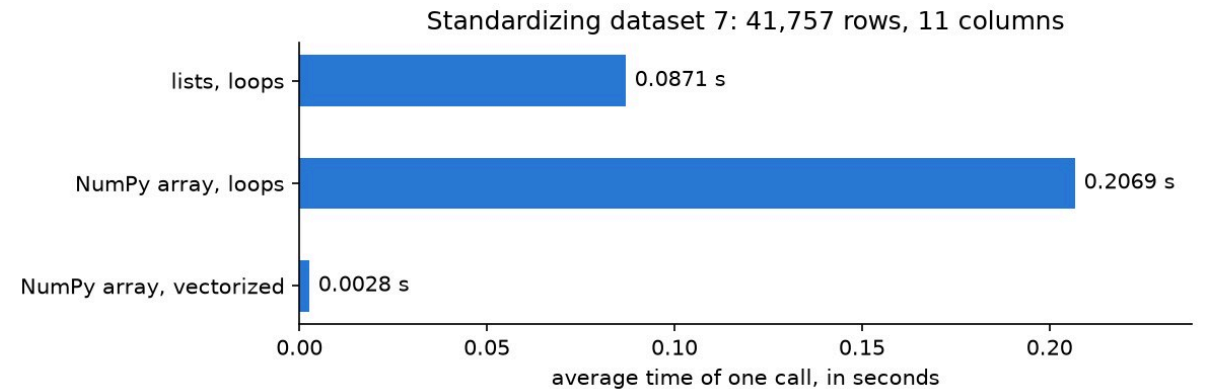
Meeting 10, Revisited

Six ideas of meeting 10 that this meeting builds on: vectorization, broadcasting, slicing, axes, word vectors, and batches.

Vectorization: a calculation without a loop

```
def standardize_array_loops(X):  
    n, m = X.shape  
    Z = np.empty((n, m))  
    for j in range(m):  
        total = 0.0  
        for i in range(n):  
            total += X[i, j]  
        mean = total / n  
        total = 0.0  
        for i in range(n):  
            total += (X[i, j] - mean) ** 2  
        std = math.sqrt(total / n)  
        for i in range(n):  
            Z[i, j] = (X[i, j] - mean) / std  
    return Z
```

```
def standardize_vectorized(X):  
    return (X - X.mean(axis=0)) / X.std(axis=0)
```



- The same arithmetic, two ways
- One line: the loop runs inside NumPy, in compiled code
- Loops over a NumPy array: the slowest of all

Broadcasting: a shape that mathematics would refuse

```
mu = X.mean(axis=0)           # one mean for each column: m numbers
sigma = X.std(axis=0)
Z = (X - mu) / sigma          # standardization, in one line
print("shape of X:", X.shape, " shape of mu:", mu.shape, " shape of X - mu:", (X - mu).shape)
v = np.array([1.0, 2.0, 3.0, 4.0])
print("v + 10 =", v + 10, " one number, added to every entry")
```

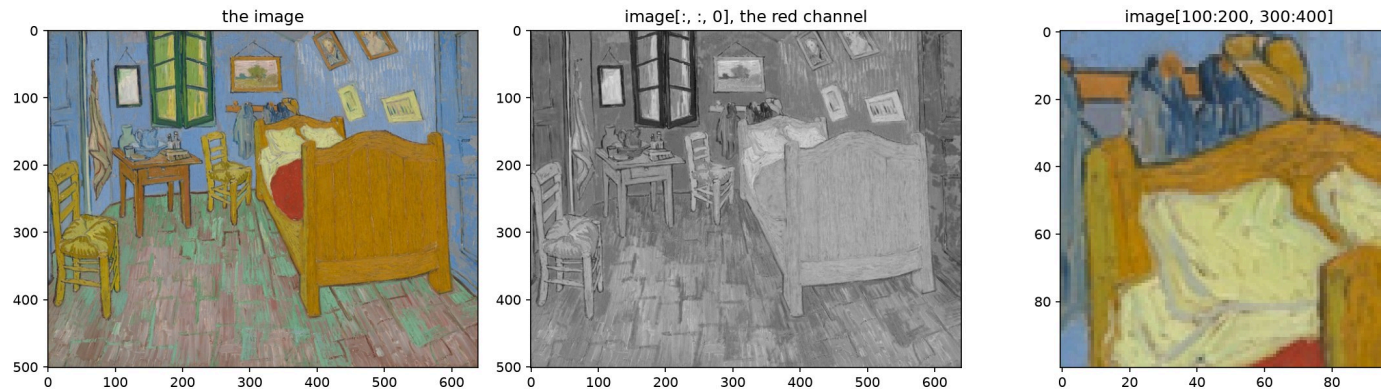
```
shape of X: (41757, 11)  shape of mu: (11,)  shape of X - mu: (41757, 11)
v + 10 = [11. 12. 13. 14.]  one number, added to every entry
```

- $X - \mu$: (n, m) minus $(m,)$
- Mathematics: write the means on every row first, $\mathbf{1}\mu^T$
- NumPy: **broadcasting**, with no copies stored
- Today: the bias of a layer, $+ b$

Slicing: a row, a column, a channel, a block

```
print("X[0]                a row, one point:          shape", X[0].shape)
print("X[:, 0]            a column, one feature:      shape", X[:, 0].shape)
print("image              H rows, W columns, 3 channels:", image.shape)
print("image[:, :, 0]      the red channel:          shape", image[:, :, 0].shape)
print("image[100:200, 300:400] a block of pixels:      shape", image[100:200, 300:400].shape)
```

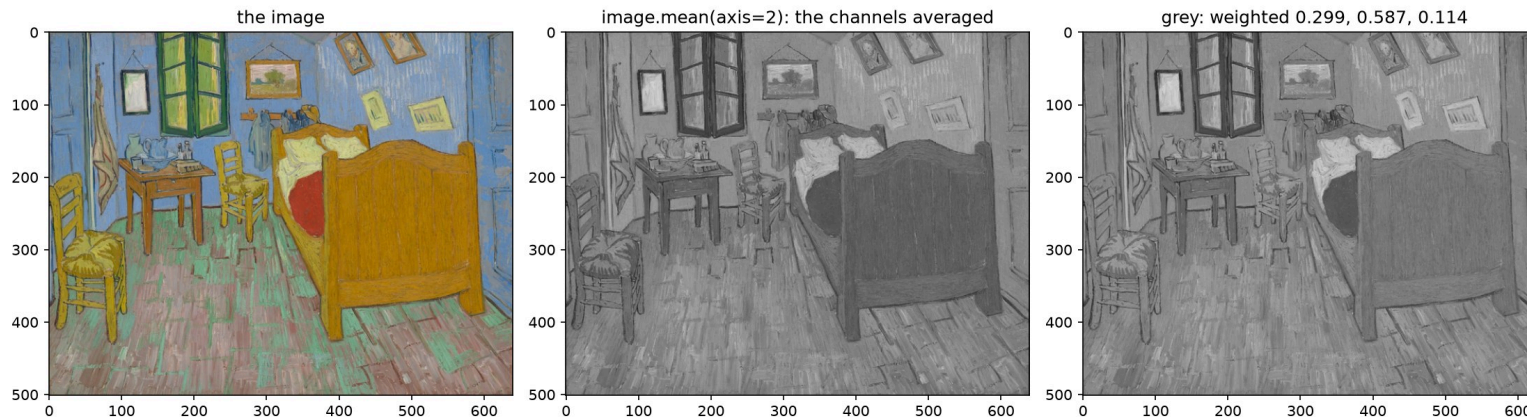
```
X[0]                a row, one point:          shape (11,)
X[:, 0]            a column, one feature:      shape (41757,)
image              H rows, W columns, 3 channels: (501, 640, 3)
image[:, :, 0]      the red channel:          shape (501, 640)
image[100:200, 300:400] a block of pixels:      shape (100, 100, 3)
```



Along an axis: the dimension that disappears

```
print("X.mean(axis=0)      down the rows: one number for each column  shape", X.mean(axis=0).shape)
print("X.mean(axis=1)      along the columns: one number for each row  shape", X.mean(axis=1).shape)
print("image.mean(axis=2)   over the three channels of each pixel      shape", image.mean(axis=2).shape)
```

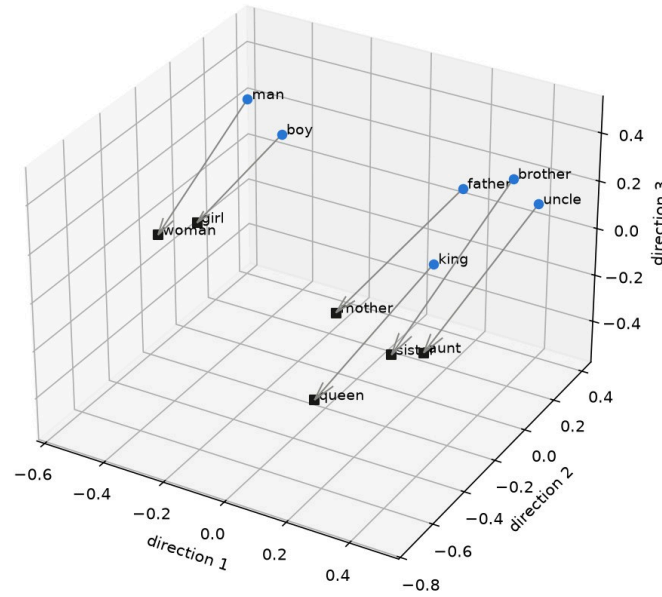
```
X.mean(axis=0)      down the rows: one number for each column  shape (11,)
X.mean(axis=1)      along the columns: one number for each row  shape (41757,)
image.mean(axis=2)   over the three channels of each pixel      shape (501, 640)
```



- The axis named is the dimension that disappears
- Today: `S.max(axis=1)`, the largest score of each row

Word vectors: arrows in a space of 300 dimensions

12 word vectors of 300 numbers, drawn in three dimensions



vector of king - man + woman, nearest words: ['queen', 'princess', 'throne']

- A word: a vector of 300 numbers. Two words: a direction between them
- Drawn in 3 of the 300 dimensions: a shadow of the arrows
- `U @ v`: one word against all 50,000, in one product

One point, then a batch

data	one point	N points
a day of weather	$(m,) = (2,)$	(N, m)
a colour image	$(H, W, 3)$	$(N, H, W, 3)$
a word	$(m,) = (300,)$	(N, m)
a text of L words	(L, m)	(N, L, m)

data	one point	N points	N
a day of weather	$(2,)$	$(6940, 2)$	6,940
a colour image	$(96, 96, 3)$	$(30, 96, 96, 3)$	30
a word	$(300,)$	$(50000, 300)$	50,000
a text of L words	$(9, 300)$	$(3, 9, 300)$	3

- N, the number of points, is the first dimension; the point's own dimensions follow

What this meeting adds

- Meeting 10: **data** stored as arrays
- Today: a **rule** that reads a point, a **neuron**, stored the same way
- The rules of a layer: the rows of a matrix W . A layer on a batch: one product
- **Act 1:** days of weather at Newark airport, then the pixels of your image, in NumPy
- **Act 2:** the same layer in PyTorch, and on a GPU
- **Ask Colab for a GPU before you run anything:** the class notebook says how